

Compiler Design Lab Programs

Name: Renee Vincent A.S.R

Reg No: 192424161

Course Code: CSA1402

13. Write a C program for implementing a Lexical Analyzer to Count the number of characters, words, and lines.

Code:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

int main() {

    char text[1000];

    int characters = 0, words = 0, lines = 0, i;

    int inWord = 0;

    printf("Enter text (single input, press Enter when done):\n");

    fgets(text, sizeof(text), stdin);

    for (i = 0; text[i] != '\0'; i++) {

        if (text[i] == '\n')

            lines++;

        if (!isspace(text[i]))

            characters++;

        if (isspace(text[i]))

            inWord = 0;

        else if (inWord == 0) {

            inWord = 1;

            words++;

        }

    }

    printf("\nCharacters: %d\n", characters);

    printf("Words: %d\n", words);

    printf("Lines: %d\n", lines);

    return 0;
```

```
}
```

Output:

Enter text : Hello world from compiler

Characters: 22

Words: 4

Lines: 0

Output Screenshot:**14. Write a C Program for code optimization to eliminate common subexpression.****Code:**

```
#include <stdio.h>

int main() {
    int a, b, c, d;
    int x_before, y_before;
    int x_after, y_after;
    printf("Enter values for a, b, c, d: ");
    scanf("%d %d %d %d", &a, &b, &c, &d);
    x_before = (a + b) - (c + d);
    y_before = (a + b) * (c + d);
    int t1 = a + b;
    int t2 = c + d;
    x_after = t1 - t2;
    y_after = t1 * t2;
    printf("\nBefore Optimization:\n");
    printf("x = (a + b) - (c + d) = %d\n", x_before);
```

```

printf("y = (a + b) * (c + d) = %d\n", y_before);
printf("\nAfter Optimization:\n");
printf("t1 = a + b = %d\n", t1);
printf("t2 = c + d = %d\n", t2);
printf("x = t1 - t2 = %d\n", x_after);
printf("y = t1 * t2 = %d\n", y_after);
return 0;
}

```

Output:

Enter values for a, b, c, d: 2 3 1 4

Before Optimization:

$$x = (a + b) - (c + d) = 0$$

$$y = (a + b) * (c + d) = 25$$

After Optimization:

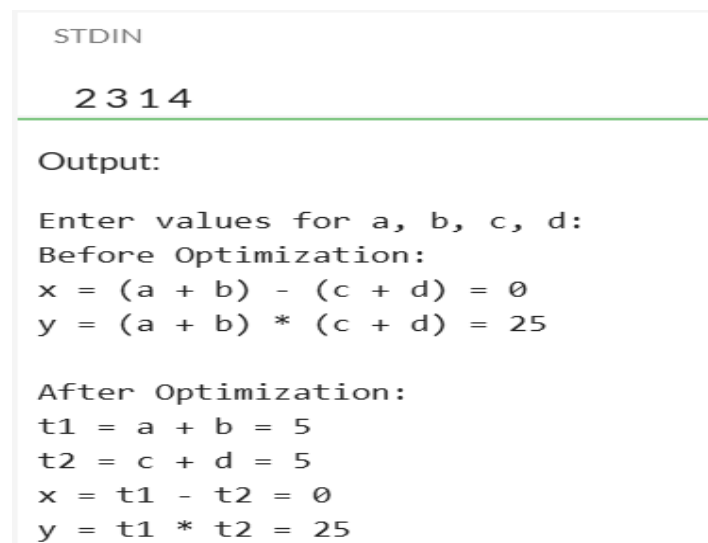
$$t1 = a + b = 5$$

$$t2 = c + d = 5$$

$$x = t1 - t2 = 0$$

$$y = t1 * t2 = 25$$

Output Screenshot:



15. Write a C program to implement the back end of the compiler.

Code:

```
#include <stdio.h>

int main() {
    char a, b, c, d;
    printf("Enter variable names (a, b, c, d): ");
    scanf(" %c %c %c %c", &a, &b, &c, &d);
    printf("\n--- Three Address Code ---\n");
    printf("t1 = %c * %c\n", c, d);
    printf("%c = %c + t1\n", a, b);
    printf("\n--- Assembly Code Simulation ---\n");
    printf("LOAD %c\n", c);
    printf("MUL %c\n", d);
    printf("STORE t1\n");
    printf("LOAD %c\n", b);
    printf("ADD t1\n");
    printf("STORE %c\n", a);
    return 0;
}
```

Output:

Enter variable names (a, b, c, d): a b c d

--- Three Address Code ---

t1 = c * d

a = b + t1

--- Assembly Code Simulation ---

LOAD c

MUL d

STORE t1

LOAD b

ADD t1

STORE a

Output Screenshot:

```
STDIN
abcd

Output:
Enter variable names (a, b, c, d):
--- Three Address Code ---
t1 = c * d
a = b + t1

--- Assembly Code Simulation ---
LOAD c
MUL d
STORE t1
LOAD b
ADD t1
STORE a
```

16. The lexical analyser should ignore redundant spaces, tabs and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Write a LEX specification file to take input C program from a .c file and count the number of characters, number of lines & number of words.

Code:

```
%{
#include <stdio.h>

int characters = 0, words = 0, lines = 0;
%}

%%

[ \t]+      ;          /* ignore spaces/tabs */
\n          { lines++; characters++; }
[A-Za-z0-9_]+ { words++; characters += yyleng; }
.           { characters += yyleng; }

%%

int main(){ yylex(); printf("Characters: %d\nWords: %d\nLines: %d\n", characters, words,
lines); return 0; }

int yywrap(){ return 1; }
```

Output:

```
#include <stdio.h>
```

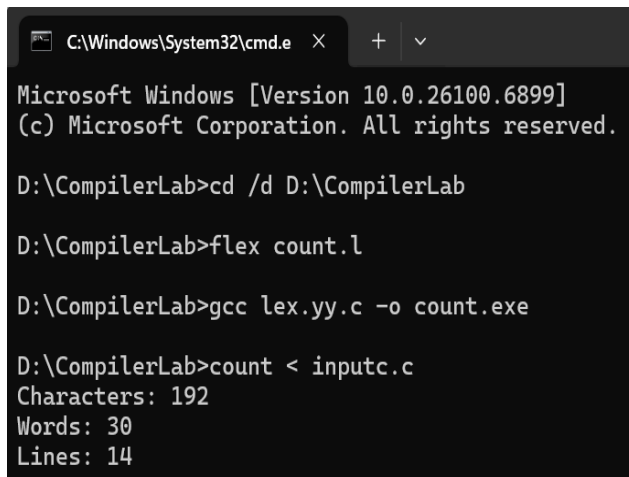
```
int main()
{
    printf("Hello");
}
```

Characters: 48

Words: 7

Lines: 6

Output Screenshot:



```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

D:\CompilerLab>cd /d D:\CompilerLab

D:\CompilerLab>flex count.l

D:\CompilerLab>gcc lex.yy.c -o count.exe

D:\CompilerLab>count < inputc.c
Characters: 192
Words: 30
Lines: 14
```

17. Write a LEX program to print all the constants in the given C source program file.

Code:

```
%{
#include <stdio.h>

%}

%%

[0-9]+(\.[0-9]+)? { printf("Constant: %s\n", yytext); }

\"([^\"]|\\.)\" ; /* ignore string literals */

.\n ; /* ignore others */

%%

int main(){ yylex(); return 0; }

int yywrap(){ return 1; }
```

Output:

```
#define PI 3.14
```

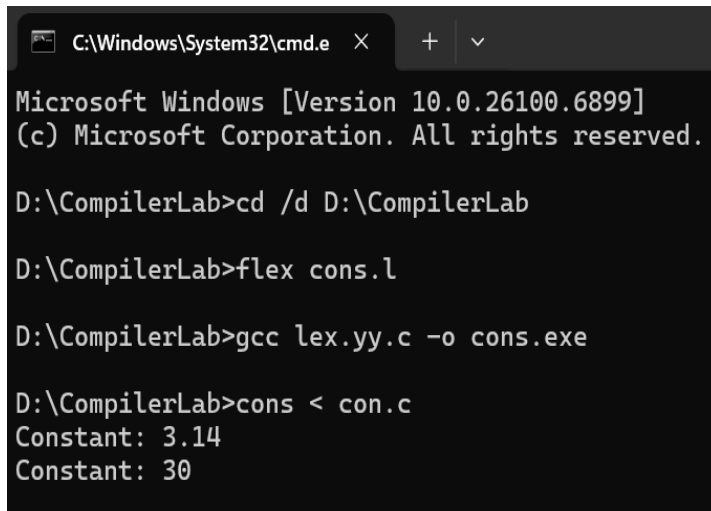
```
int x = 30;
```

```
printf("val %d", x);
```

Constant: 3.14

Constant: 30

Output Screenshot:



```
C:\Windows\System32\cmd.e × + v
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

D:\CompilerLab>cd /d D:\CompilerLab

D:\CompilerLab>flex cons.l

D:\CompilerLab>gcc lex.yy.c -o cons.exe

D:\CompilerLab>cons < con.c
Constant: 3.14
Constant: 30
```